VULNERABILITY RESEARCH

Unveiling the GoFetch Vulnerability: A Side-Channel Attack on Apple M-Series Prefetchers

Eisha Khan

VULNERABILITY RESEARCH PROJECT

ABSTRACT

GoFetch is a microarchitectural side-channel attack, disclosed in 2024, that extracts secret cryptographic keys from constant-time software by abusing the data memory-dependent prefetcher (DMP) built into Apple M-series processors. Because the DMP treats in-memory data that resembles a pointer as an address and speculatively fetches it, it can be manipulated into leaking secret-dependent information through the CPU cache — defeating constant-time programming, the standard defense against timing side channels. This paper explains how the DMP works, why it breaks the constant-time paradigm, how the attack recovers keys through chosen inputs and cache-timing analysis, which processors and algorithms are affected, and what software mitigations are available. Findings are drawn from the original research published at gofetch.fail and the 2024 USENIX Security paper by Chen et al.

1. Introduction

Timing side-channel attacks recover secrets by measuring how long cryptographic operations take. For years, the primary defense has been *constant-time programming*: writing code so that neither its execution time, its branches, nor its memory-access addresses depend on secret data. If an implementation never uses a secret as an address and never branches on a secret, an attacker watching timing or cache behavior should learn nothing about the key.

GoFetch challenges this assumption at the hardware level. It demonstrates that a modern performance optimization — the data memory-dependent prefetcher present in Apple silicon — can turn ordinary program data into memory addresses from inside the processor, without the program ever asking it to. As a result, even correctly written constant-time code can be coerced into leaking key material. This paper examines the vulnerability, the mechanism behind it, and its practical impact.

2. Background

Prefetchers and the Data Memory-Dependent Prefetcher (DMP)

A prefetcher is a hardware feature that predicts which memory a program will need next and loads it into the cache ahead of time to reduce latency. Traditional prefetchers base these predictions on *access patterns* — for example, detecting that a loop is walking sequentially through an array. A data memory-dependent prefetcher goes further: it inspects the actual *contents* of memory. If a value sitting in the

cache looks like it could be a pointer, the DMP will speculatively dereference it and fetch whatever it appears to point to. In effect, the DMP turns data into addresses on the program's behalf.

Constant-Time Cryptography

The constant-time paradigm forbids using secret-dependent data as a memory address, precisely because an attacker who can observe cache activity could otherwise infer the secret from which addresses were touched. GoFetch's core insight is that the DMP violates this guarantee from below: the software never uses the secret as an address, but the prefetcher does it anyway when a secret-dependent value happens to resemble a pointer. The separation between data and addresses that constant-time code carefully maintains is undone inside the memory system.

3. How the GoFetch Attack Works

GoFetch is a chosen-input attack. The attacker feeds carefully crafted inputs into a victim's cryptographic operation such that an intermediate value only *looks like a valid pointer* when a guessed portion of the secret key is correct. The attacker then watches, through cache-timing measurement, whether the DMP performed a dereference:

- If the DMP activates and fetches the "pointed-to" address, the corresponding cache line becomes fast to access — a measurable signal that the crafted value resembled a pointer, confirming the key-bit guess.
- If the DMP does not activate, no such signal appears, indicating the guess was wrong.

By repeating this guess-and-verify process, the attacker recovers the key a few bits at a time until the entire secret is reconstructed. The researchers illustrate the principle with a constant-time conditional swap: a pointer is placed in one buffer and a target value in another, and the presence or absence of a DMP dereference reveals whether the secret-dependent swap occurred. The measurement itself relies on standard cache side-channel techniques, using eviction sets to observe which lines the prefetcher pulled in.

> **Why it matters:** the attack requires no software bug in the cryptographic library and no secret-dependent code. The leak originates in the processor's prefetcher, which means a perfectly constant-time implementation is still vulnerable when run on affected hardware.

4. Affected Systems and Demonstrated Impact

The researchers reverse-engineered the DMP behavior on Apple silicon and mounted end-to-end key-extraction attacks on M1-based machines, confirming that the M2 and M3 generations exhibit similar exploitable behavior. The A14 generation is also affected. The same class of prefetcher exists in Intel's 13th-generation "Raptor Lake" processors, but Intel's more restrictive implementation prevented the attack in the researchers' testing.

Crucially, the attack was demonstrated against real, widely used constant-time cryptographic implementations spanning both classical and post-quantum schemes:

Algorithm	Category
OpenSSL Diffie-Hellman Key Exchange	Classical
Go RSA Decryption	Classical
CRYSTALS-Kyber	Post-Quantum (KEM)
CRYSTALS-Dilithium	Post-Quantum (Signatures)

Apple was notified of the findings in December 2023. Because the weakness is built into the silicon's prefetcher, it cannot be patched directly in already-shipped chips; defenses must be implemented in software, and they carry a performance cost.

5. Mitigations

Since the flaw cannot be fixed in hardware on affected devices, mitigation falls to cryptographic libraries and developers. Documented approaches include:

- **Cryptographic blinding / masking:** randomizing the values involved in a computation (for example, ciphertext blinding for Kyber) so that intermediate values no longer correlate with the secret in a way the DMP can leak.
- **Running sensitive code on efficiency cores:** Apple's efficiency ("Icestorm") cores do not exhibit the same aggressive DMP behavior, so performing cryptographic operations there avoids the leak — at the expense of throughput.
- **Hardware timing controls where available:** newer chips expose a Data-Independent Timing (DIT) control that can constrain prefetcher-related behavior for security-critical code.

Each of these defenses reduces performance, which is why commentators noted that mitigating GoFetch may impose a meaningful cost on the very cryptographic operations it protects.

6. Discussion

GoFetch is significant because it undermines a foundational defensive technique rather than a single library. Constant-time programming has been the backbone of side-channel-resistant cryptography, and this work shows that hardware optimizations can silently invalidate its guarantees. The result is especially relevant as post-quantum algorithms such as Kyber and Dilithium move toward widespread deployment: their constant-time implementations inherit the same exposure on affected hardware. More broadly, GoFetch highlights the ongoing tension between aggressive microarchitectural performance features and the security assumptions software relies on, and argues for treating data-dependent prefetching as a first-class consideration in both cryptographic engineering and processor design.

7. Conclusion

GoFetch demonstrates the first end-to-end key-extraction attacks that exploit the Apple M-series data memory-dependent prefetcher to break constant-time cryptographic implementations. By coercing the prefetcher into dereferencing secret-dependent values and observing the results through the cache, an attacker can recover keys from classical and post-quantum algorithms alike, on hardware that cannot be patched. The available mitigations are software-based and impose performance overhead. The vulnerability underscores that defending cryptographic code requires reasoning not only about the software's own behavior but also about the microarchitectural features executing beneath it.

8. References

- Chen, B., Wang, Y., Shome, P., Fletcher, C., Kohlbrenner, D., Paccagnella, R., & Genkin, D. (2024). *GoFetch: Breaking Constant-Time Cryptographic Implementations Using Data Memory-Dependent Prefetchers.* 33rd USENIX Security Symposium. https://gofetch.fail/
- USENIX Security '24 — Paper and proceedings. usenix.org/conference/usenixsecurity24/presentation/chen-boru
- The Hacker News (2024). *New "GoFetch" Vulnerability in Apple M-Series Chips Leaks Secret Encryption Keys.*
- BleepingComputer (2024). *New GoFetch attack on Apple Silicon CPUs can steal crypto keys.*